GROUP I - PROJECT SUBMISSION REPORT

Project Title: Time-Table Generator	**Group Id:** I

Student ID:	Student Name:
202301049	Ramoliya Shivam S.
202301143	Patel Pranshu
202301202	Vadodariya Jenish
202301442	Ramoliya Shubham S.
202301461	Ayush B. Chovatiya

I. Brief Description of the Project

The Time-Table Generator is a software system designed to automatically generate a conflict-free university timetable. It assigns courses to time slots and rooms while satisfying constraints such as faculty availability, room capacity, batch schedules, and course requirements. The system also optimizes scheduling by minimizing idle time, distributing lectures evenly, and ensuring consistency.

II. Process Model Chosen and Justification

The Agile model is chosen for the Time-Table Generator project because the system requirements and constraints are not fully defined at the beginning and are expected to evolve during development. Since the project involves complex constraint handling (faculty, room, batch, and time-slot conflicts), an iterative approach allows incremental implementation and continuous validation of these rules. Agile enables us to build the system in small working increments, present a Proof of Concept after each sprint, gather feedback, and refine the logic accordingly. This flexibility makes Agile more suitable than rigid models like RAD for developing a constraint-based scheduling system.

III. Function Point Analysis :

In project management, PERT stands for Program Evaluation and Review Technique. It is a
three-point estimating method used to calculate a realistic expected duration or cost for project tasks using:

- Optimistic (O): Best-case scenario,
- Most Likely (M): Realistic scenario,
- Pessimistic (P): Worst-case scenario.

The PERT formula is:

Expected Time (TE) = (O + 4M + P) / 6

This weighted average gives more importance to the Most Likely estimate (4× weight), based on the Beta Distribution, providing a more accurate forecast than a simple average.

Number External Inputs

Sl.No.	Details	Optimistic	Likely	Pessimistic	Est. Count	Weight	FP Count
1.	Course Management (Add/update/Delete)	2	3	5	3	4	12
2.	Faculty Management	2	3	4	3	4	12
3.	Batch/Section Management	2	3	4	3	3	9
4.	Room Management	2	3	4	3	3	9
5.	Faculty Availability & Load Entry	1	2	4	2	6	12
6.	Course Requirement Entry	1	2	3	2	4	8
7.	Working Days & Time Slot Setup	1	2	3	2	4	8
8.	Generate Timetable Command	1	1	2	1	6	6
	Total						**79**

Number of External Outputs

Sl.No.	Details	Optimistic	Likely	Pessimistic	Est. Count	Weight	FP Count
1.	Batch-wise TimeTable	1	2	3	2	5	10
2.	Faculty-Wise Timetable	1	2	3	2	5	10
3.	Room-Wise Allocation	1	1	2	1	4	4
4.	Conflict Report	1	1	2	1	7	7
5.	Faculty Load Report	1	2	3	2	5	10
	Total						**41**

Number of External Enquires

Sl.No,	Details	Optimistic	Likely	Pessimistic	Est. Count	Weight	FP Count
1.	View Faculty Availability	1	2	3	2	4	8
2.	View Room Availability	1	2	3	2	4	8
3.	View Course Details	1	1	2	1	3	3
4.	Check Free Time Slot	2	2	4	2	6	12
	Total						**31**

Number of Internal Logical Files

Sl.No,	Details	Optimistic	Likely	Pessimistic	Est. Count	Weight	FP Count
1.	Course File	1	1	2	1	7	7
2.	Faculty File	1	1	2	1	7	7
3.	Batch File	1	1	2	1	7	7
4.	Room & Lab File	1	1	2	1	7	7
5.	Tile Slot File	1	1	2	1	7	7
6.	Faculty Availability File	1	1	2	1	10	10
7.	Generated Timetable File	1	1	2	1	15	15
	Total						**60**

- **Calculation :**

Component	Count	FP Total
External Inputs (EI)	9	79
External Output (EO)	5	41
External Inquiries (EQ)	4	31
Internal Logic Files (EIF)	7	60
Total UFP		**211**

Value Adjustment Factor (VAF) Assessment

A default VAF of 1.00 assumes a system of baseline, average complexity across all operational parameters. For a modern, interactive web-based scheduling application, standard environmental and technical complexities require a formal evaluation of the 14 General System Characteristics (GSCs) on a scale of 0 (Not Present) to 5 (Strong Influence).

General System Characteristics (GSC) Scoring Table

No.	Characteristic	Score (0-5)	Justification
1	**Data Communications**	4	Continuous API communication required between the web frontend and backend services.
2	**Distributed Data Processing**	3	Application logic and database management are separated into distinct architectural layers.
3	**Performance**	3	Requires standard web responsiveness; no extreme microsecond latency requirements.
4	**Heavily Used Configuration**	2	Deployed on standard cloud infrastructure; does not require highly specialized hardware.
5	**Transaction Rate**	3	Subject to moderate seasonal spikes (e.g., beginning of academic semesters).

6	**Online Data Entry**	5	The system is entirely interactive, relying heavily on web forms for data input.
7	**End-User Efficiency**	4	High priority for intuitive UI/UX to ensure faculty and administrators can navigate easily.
8	**Online Update**	4	Timetables, allocations, and availability must be updated and reflected in real-time.
9	**Complex Processing**	4	Generating conflict-free timetables requires intensive mathematical and logical constraint checking.
10	**Reusability**	3	Codebase is structured with modular, reusable frontend components and backend services.
11	**Installation Ease**	2	No end-user installation required (web-based), though server deployment involves standard setup.
12	**Operational Ease**	3	Requires standard administrative oversight, automated backups, and log monitoring.
13	**Multiple Sites**	2	Designed primarily for a single institution, though accessible remotely by multiple users.

14	**Facilitate Change**	3	Architecture must accommodate evolving university rules, shifting batch structures, and flexible time slots.

Final Function Point Calculations

Based on the calculated Unadjusted Function Point (UFP) count of **211** and the GSC evaluation above, the final system size is calculated as follows:

1. Total Degree of Influence (TDI)

The sum of all 14 GSC scores.

- TDI = 4 + 3 + 3 + 2 + 3 + 5 + 4 + 4 + 4 + 3 + 2 + 3 + 2 + 3
- **TDI = 45**

2. Value Adjustment Factor (VAF)

Using the standard IFPUG formula based on the TDI.

- VAF = 0.65 + (0.01 * TDI)
- VAF = 0.65 + (0.01 * 45)
- **VAF = 1.10**
  (A VAF of 1.10 indicates the system's environmental complexity is 10% higher than average).

3. Adjusted Function Points (AFP)

The final, adjusted size of the software system.

- AFP = UFP * VAF
- AFP = 211 * 1.10
- **AFP = 232.1**

Final Estimated Size: 232 Function Points

IV. Team Composition and roles :

Sr No.	Name	Roles & Responsibility
1	Shivam	Frontend Developer & Project Coordinator. Manages Agile sprints, task allocation, and overall system architecture. Designs database schema and monitors project progress, SDD document.
2	Shubham	Backend Developer. Develops timetable generation logic. Implements constraint handling (faculty, room, batch conflicts) and optimizes performance.
3	Ayush	Documented Risk Information Sheet, manages DB tables and relationships, project estimation, requirement gathering, documentation
4	Jenish	Sprint development, Design Gantt Chart, monitors project progress, handles data population inside database, helping in testing, requirement gathering, project planning and estimation.
5	Pranshu	Testing & Program Logic. Writes test cases, performs unit and integration testing, timetable generation logic, and system design

V. System Requirements Specification

1. Project Objective:

The Time-Table Generator is a system that automatically creates a university timetable. It assigns courses to available time slots and classrooms while ensuring that no conflicts occur. The main objective of this system is to generate a valid and optimized timetable by satisfying all required constraints such as faculty availability, classroom capacity, and student batch schedules. This system reduces manual effort, saves time, and improves scheduling accuracy.

2. Scope of the System:

1. This system is designed for universities and colleges where multiple courses, faculty members, classrooms, and student batches need to be scheduled efficiently.

2. The system will take necessary input data and generate a complete timetable automatically. It ensures that there are no conflicts between faculty, rooms, and student batches. In case a timetable cannot be generated due to strict constraints, the system will detect and report conflicts.
3. Additionally, the system aims to optimize scheduling by reducing idle time and distributing lectures evenly across the week.

3. Input Requirements

Before generating the timetable, the system requires the following inputs:

1. **Course Information**
   Course name, number of lectures per week, and lecture slot.
2. **Faculty Information**
   List of faculty members along with their available time slots and teaching limits.
3. **Student Batch Information**
   List of batches/sections and number of students in each batch.
4. **Room Information**
   Classrooms/labs with their seating capacity and type.
5. **Time Slot Information**
   Working days, available time slots, and break periods (e.g., lunch break).

This input data helps the system understand all scheduling requirements and constraints.

4. Functional Requirements (FRs) :

1. **Input Course Details**
   a. System shall allow entry of course name, code, credits, and assigned faculty.
2. **Faculty Management**
   a. The system shall store faculty availability and preferences.
   b. The system shall prevent scheduling conflicts for faculty.
3. **Room Management**
   a. The system shall manage room details (capacity, type like lab/classroom).
   b. The system shall assign rooms based on capacity and requirements.
4. **Batch/Student Group Handling**
   a. The system shall manage multiple batches/sections.

   b. The system shall avoid overlapping classes for the same batch.

5. **Time Slot Allocation**
   a. The system shall define available time slots and working days.
   b. The system shall assign courses to valid time slots.
6. **Conflict Detection & Resolution**
   a. The system shall detect clashes (faculty, room, batch).
   b. The system shall ensure the generated timetable is conflict-free.
7. **Constraint Handling**
   a. System shall support constraints like:
      i. Faculty availability
      ii. Room capacity
      iii. Lab requirements
      iv. Preferred time slots
8. **Optimization Features**
   a. The system shall minimize idle time for faculty and students.
   b. The system shall distribute lectures evenly across the week.
9. **Timetable Export**
   a. The system shall allow export (PDF/Excel).

5. Non-Functional Requirements (NFRs)

1. **Performance**
   a. The system should generate timetables within acceptable time (e.g., < 10 seconds for medium data).
2. **Scalability**
   a. The system should handle an increasing number of courses, rooms, and batches efficiently.
3. **Usability**
   a. Interfaces should be user-friendly and easy to operate.
   b. Minimal training required.
4. **Reliability**
   a. The system should consistently generate correct, conflict-free timetables.
5. **Maintainability**
   a. The system should be easy to update (e.g., adding new constraints).
6. **Flexibility**
   a. The system should support changes in academic structure or scheduling rules.

6. Output Requirements

After processing all inputs, the system will generate:

1. **Batch-wise Timetable**
   Schedule of classes for each student batch.
2. **Faculty-wise Timetable**
   Schedule of classes assigned to each faculty member.
3. **Room Allocation Schedule**
   Details of classroom/lab assigned for each lecture.
4. **Conflict/Error Report**
   If constraints cannot be satisfied, the system will display conflicts.

The output will be presented in the form of structured tables or Excel sheets for easy viewing and usage.

VI. System Design Document

1. Introduction

1.1 Purpose

This document provides a detailed design of the Time-Table Generator system, including architecture, data structures, modules, and interfaces. It serves as a blueprint for development and maintenance.

1.2 Scope

The system will automatically generate a conflict-free timetable
Consider constraints:

a. Faculty availability
b. Room capacity
c. Batch schedules
d. Course requirements

Optimize scheduling by:

e. Minimizing idle time
f. Distributing lectures evenly

2. System Overview

The Time-Table Generator is a constraint-based scheduling system that:

Accepts input data (courses, faculty, rooms, etc.) -> Applies scheduling logic -> Generates multiple timetable views

3. System Architecture

3.1 Architecture Type: 3-Tier Architecture

Presentation Layer (UI)

↓

Application Layer (Business Logic)

↓

Data Layer (Database)

4. Module Design (Component Design)

4.1 Timetable Generation Module (Core)

- Applies constraints:
    - No faculty conflict
    - No room conflict
    - No batch overlap
    - No back to back lecture for faculty
- Uses algorithms:
    - Greedy
    - Backtracking
    - Genetic Algorithm (optional)

4.2 Reporting Module

- Generate:
    - Batch-wise timetable
    - Faculty-wise timetable
    - Room allocation
    - Conflict reports

5. Data Design (Annexure – II)

Faculty

Field	Type	Key Status
faculty_id	BIGSERIAL	Primary Key
short_name	TEXT	Unique NOT NULL
full_name	TEXT	
created_at	TIMESTAMP	NOT NULL DEFAULT now()

Course

Field	Type	Key Status
course_id	Big SERIAL	Primary Key
code	TEXT	Unique NOT NULL
title	TEXT	NOT NULL
lecture_hours	INT	NOT NULL
tutorial_hours	INT	NOT NULL DEFAULT 0
practical_hours	INT	NOT NULL DEFAULT 0
credits	NUMERIC (5, 2)	NOT NULL
course_type	TEXT	NOT NULL
created_at	TIMESTAMP	

StudentBatch

Field	Type	Key Status
batch_id	BIGSERIAL	Primary Key
batch_code	TEXT	Unique NOT NULL
program	TEXT	NOT NULL
semester	INT	NOT NULL
batch_size	INT	NOT NULL
created_at	TIMESTAMP	NOT NULL

Room

Field	Type	Key Status
room_id	BIGSERIAL	Primary Key
room_code	TEXT	Unique NOT NULL
capacity	INT	NOT NULL
room_type	TEXT	NOT NULL DEFAULT 'Lecture Hall'
created_at	TIMESTAMP	NOT NULL

TimeMatrix

Field	Type	Key Status
slot_id	BIGSERIAL	Primary Key
day_of_week	TEXT	NOT NULL
start_time	TIME	NOT NULL

end_time	TIME	NOT NULL
slot_group	TEXT	NOT NULL DEFAULT 'Teaching'
is_blackout	BOOLEAN	NOT NULL DEFAULT FALSE

FacultyCourseMap

Field	Type	Key Status
assignment_id	BIGSERIAL	Primary Key
faculty_id	BIGINT	Foreign Key NOT NULL
course_id	BIGINT	Foreign Key NOT NULL

BatchCourseMap

Field	Type	Key Status
batch_id	BIGINT	PK + FK
course_id	BIGINT	PK + FK
faculty_id	BIGINT	Foreign Key NOT NULL

ConstraintConfig

Field	Type	Key Status
config_id	BIGSERIAL	Primary Key
key	TEXT	Unique NOT NULL
value_json	JSON	NOT NULL
updated_at	TIMESTAMP	NOT NULL

ScheduleRun

Field	Type	Key Status
run_id	BIGSERIAL	Primary Key
label	TEXT	NOT NULL
source_csv	TEXT	
status	TEXT	NOT NULL DEFAULT 'draft'
notes	TEXT	
created_at	TIMESTAMP	

MasterTimetable

Field	Type	Key Status
timetable_id	BIGSERIAL	Primary Key
run_id	BIGINT	Foreign Key NOT NULL
assignment_id	BIGINT	Foreign Key NOT NULL
batch_id	BIGINT	Foreign Key NOT NULL
room_id	BIGINT	Foreign Key NOT NULL
slot_id	BIGINT	Foreign Key NOT NULL
lecture_index	INT	
merged_batch_ids	TEXT	
is_merged	BOOLEAN	DEFAULT FALSE

ConflictReport

Field	Type	Key Status
report_id	BIGSERIAL	Primary Key
run_id	BIGINT	Foreign Key NOT NULL
severity	TEXT	NOT NULL
category	TEXT	NOT NULL
detail	TEXT	NOT NULL
created_at	TIMESTAMP	NOT NULL

ER Diagram : **ER Diagram Link**

6. Interface Design (Annexure – III)

Input Forms

- Course Entry Form
- Faculty Entry Form
- Room Entry Form
- Generate Timetable Button

Output Screens

- Batch-wise timetable view
- Faculty-wise timetable view
- Room-wise allocation
- Conflict report screen

7. Algorithm Design (Core Logic)

Steps:

1. Load all input data (courses, faculty, rooms, slots)
2. Sort courses based on priority
3. For each course:
    - Assign free room

    - Assign free time slot
4. Validate constraints:
    - No overlap
    - Availability satisfied
5. If conflict:
    - Backtrack and retry
6. Save final timetable

8. Security Design

- Input validation to prevent invalid data

9. Performance & Scalability

- Efficient scheduling algorithms
- Database indexing
- Scalable for large number of courses and users

10. Error Handling

- Conflict detection system
- Missing data validation
- Error messages for invalid input

11. Deployment Design

- Platform: Web-based system
- Server: Localhost / Cloud
- Database: PostgreSQL

12. Assumptions & Constraints

- Fixed number of time slots per day
- Faculty availability is predefined
- Room capacity is sufficient for batches
- All lecture duration are same

13. Future Enhancements

- AI-based timetable optimization
- Drag-and-drop manual editing
- Mobile application support
- Real-time conflict visualization

VII. Sprint Documentation

Sl.No.	Sprint Description	Document _Reference
1.	Sprint 1: Requirement Analysis & Planning	Includes process model selection and justification, requirement gathering, documentation, and function point analysis to estimate system complexity.
2.	Sprint 2: System Design	Covers preparation of the System Design Document (SDD), including overall system architecture, module design, and data flow structure.
3.	Sprint 3: Core Development	Focuses on architecture setup, database implementation, and development of the basic scheduling algorithm.
4.	Sprint 4: Improvement, Testing & Finalization	Includes code improvement, optimization, testing, bug fixing, and final documentation for project completion.

VIII. Programs and the logics applied

1. Overview of the Program

- The system is a Python-based automatic timetable generator.
- It reads course data from CSV files and processes it to generate a 5×5 weekly timetable.
- Courses are grouped into slots and then scheduled into time periods.
- The system generates multiple outputs:
    - Global timetable (slot-based)
    - Program-wise timetable
    - Faculty-wise timetable
- The design follows a modular approach for better maintainability and scalability.

2. Driver Program Logic (main.py)

- The execution starts from the main driver program.
- The system creates or accesses input/output directories.
- It takes user input for the CSV file containing course data.
- Calls read_courses_from_csv() to load course details into objects.

- Calls build_slots(courses) to group courses into slots.
- Generates timetable using make_timetable(slots).
- Saves outputs:
    - Slots file
    - Timetable file
- Provides additional features:
    - Program-wise timetable generation
    - Faculty-wise timetable generation

3. Data Structure Logic (models.py)

3.1 Course Class

- Stores all course-related information:
    - Course code, name
    - Lecture count
    - Course type (core/elective)
    - Faculty, program, semester
- Uses a helper property is_core:
    - Filters only core courses for scheduling

3.2 Slot Class

- Represents a group of courses scheduled at the same time.
- Each slot contains:
    - Unique slot ID
    - Lecture length (1, 2, or 3)
    - List of courses
- Maintains constraints using:
    - _prog_sem_filled → prevents same program-semester clash
- Core logic methods:
    - can_add_course_group():
        - Ensures same lecture length
        - Prevents faculty conflicts
        - Avoids duplicate program-semester pairs
    - add_course_group():
        - Adds valid course groups into slot

4. Input and Output Logic (io_utils.py)

4.1 Input Processing

- Read CSV file using read_courses_from_csv().
- Extracts:
  - Course details
  - Lecture count from L-T-P-C format
  - Semester as integer
- Converts each record into a Course object.

4.2 Output Generation

- Slots File:
  - Displays grouped courses per slot
- Timetable File:
  - Represents a 5×5 grid (days × periods)
  - Uses slot IDs (M1, M2, etc.)
- Program-wise Timetable:
  - Filters courses based on program and semester
- Faculty-wise Timetable:
  - Displays schedule for individual faculty

5. Slot Generation Logic (slot_generator.py)

5.1 Requirement Calculation

- Counts number of core courses for each:
  - Program
  - Semester
- Determines number of:
  - 1-lecture slots
  - 2-lecture slots
  - 3-lecture slots
- Uses maximum requirement across all combinations

5.2 Slot Creation

- Creates slots in order:
  - First 3-lecture slots
  - Then 2-lecture slots
  - Then 1-lecture slots
- Assigns unique slot IDs

5.3 Course Grouping Logic

- Groups courses by course code
- Assigns groups into slots based on:
  - Matching lecture length
  - Core course requirement
  - No faculty overlap
  - No program-semester conflict
- Uses can_add_course_group() to validate constraints
- Ensures each course is assigned only once

6. Timetable Generation Logic : Document for program and logic applied

IX. Gantt Chart : Gantt Chart Link

X. Risk Information Sheets : Risk Information Sheets Link